Text Large Model + YOLOv11 + Intelligent Garbage Classification（Voice Version）

Before running the function, you need to close the App and large programs. For the closing method, refer to [4. Preparation] - [1. Manage APP control services].

1. Function Description

After the program starts, give the large model a garbage sorting instruction. After thinking, the large model will control the robotic arm to sort the garbage label code blocks on the table according to requirements.

2. Startup

After entering the Docker container, enter the following commands in the Docker terminal.

```
ros2 launch largemodel largemodel_control.launch.py
```

Garbage sorting has the following types of instructions:

- Sort a single type of garbage: for example, sort toxic garbage
- Keep a certain type of garbage: for example, sort all garbage except recyclable garbage
- Sort all garbage on the desktop: for example, clean up all garbage on the desktop

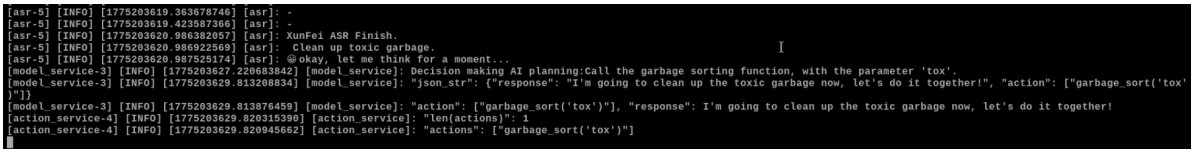
2.1. Sorting a Single Type of Garbage

2.1.1. Startup

After waking up the voice module, input the voice command, for example:

```
Clean up toxic garbage
```

Wait for the large model to think and reply, as shown in the figure below:



The robotic arm will grab the toxic garbage label code blocks on the desktop.

2.1.2. Task Planning

Call garbage_sort('tox') to sort toxic garbage.

2.1.3. Core Source Code Analysis

The garbage_sort function is located at:
`LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def garbage_sort(self, type_):
    Arm.Arm_serial_servo_write6(90,120,10,10,90,30,1000)
    time.sleep(1.0)
    type_ = type_.strip("'\"")  # Remove single and double quotes
```

```
    if type_ == "rec":
        target_type_ = float(1)
    elif type_ == "tox":
        target_type_ = float(2)
    elif type_ == "wet":
        target_type_ = float(3)
    elif type_ == "dry":
        target_type_ = float(4)
    else:
        self.get_logger().info(
            "Fatal ERROR:Incorrect type_ input,Does the AI output not meet
expectations?"
        )
        self.action_status_pub(
            "garbage_sort_failed", type_=type_
        )
        return
    # Start the robotic arm grasping program
    cmd1 = "ros2 run dofbot_pro_yolov11 yolov11_sortation"
    # Start the image conversion program
    cmd2 = "ros2 run dofbot_pro_yolov11 msgToimg"
    # Start the garbage recognition program
    cmd3 = f"ros2 run largemodel_arm yolov11_ALM --ros-args  -p target_type:=
{target_type_:.1f}"

    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=grasp_desktop_garbage_sort",
            "--",
            "bash",
            "-c",
            f"{cmd1}; exec bash",
        ]
    )

    subprocess.Popen(
        [
            "gnome-terminal",
            "+--title=detect_img",
            "--",
            "bash",
            "-c",
            f"{cmd2}; exec bash",
        ]
    )

    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=garbage_detect",
            "--",
            "bash",
            "-c",
            f"{cmd3}; exec bash",
        ]
    )
```

Mainly analyze the garbage recognition program `yolov11_ALM`, source code path: `LargeModel_ws/src/largemodel_arm/largemodel_arm/yolov11_ALM.py`

```
# Get the garbage type to be sorted through command line parameters
self.declare_parameter('target_type', 1.0)
self.TargetType =
int(self.get_parameter('target_type').get_parameter_value().double_value)
# Modify self.waste according to the value of self.TargetType, which contains all
garbage names of this type
if self.TargetType == 1:
    self.waste = ['Newspaper','Zip_top_can','Book','Old_school_bag']
elif self.TargetType == 2:
    self.waste =
['Syringe','Expired_cosmetics','Used_batteries','Expired_tablets']
elif self.TargetType == 3:
    self.waste = ['Fish_bone','Egg_shell','Apple_core','Watermelon_rind']
elif self.TargetType == 4:
    self.waste =
['Toilet_paper','Peach_pit','Cigarette_butts','Disposable_chopsticks']

# Initialize a YOLO object detection model instance, load pre-compiled TensorRT
engine file (best.engine), and specify the model task type as object detection
(detect)
self.yolo_model =
YOLO("/home/jetson/dofbot_pro_ws/src/dofbot_pro_yolov11/dofbot_pro_yolov11/best.
engine", task='detect')
# Create a subscriber to subscribe to the converted image data topic
self.image_sub =
self.create_subscription(ImageMsg,"/image_data",self.image_sub_callback,qos_prof
ile=1)

# In the callback function image_sub_callback
# Use YOLO11 for object detection
results = self.yolo_model(self.img, save=False, verbose=False)
boxes = results[0].boxes
# Traverse detection results
for box in boxes:  # detections per image
    x_min, y_min, x_max, y_max = map(int, box.xyxy[0])
    class_id = int(box.cls)
    confidence = float(box.conf)
    label = f"{self.yolo_model.names[class_id]} {confidence:.2f}"
    # Calculate center position
    center_x = (x_min + x_max) // 2
    center_y = (y_min + y_max) // 2
    # Extract the information of the currently detected label code and assign it
to center, including center coordinates and label code name
    center = Yolov11Detect()
    center.centerx = float(center_x)
    center.centery = float(center_y)
    center.result = str(self.yolo_model.names[class_id])
    # If the currently detected label code name is in the self.waste list, it
means it is the target sorting object, publish the sorting topic flag and label
code position information
    if center.result in self.waste:
        start_flag = Bool()
        start_flag.data = True
        self.pub_SortFlag.publish(start_flag)
```

```
            self.detect_flag = True
            self.cnt = self.cnt + 1
            print("Found the target.")
            self.pubDetect.publish(center)
            self.start_flag = False
```

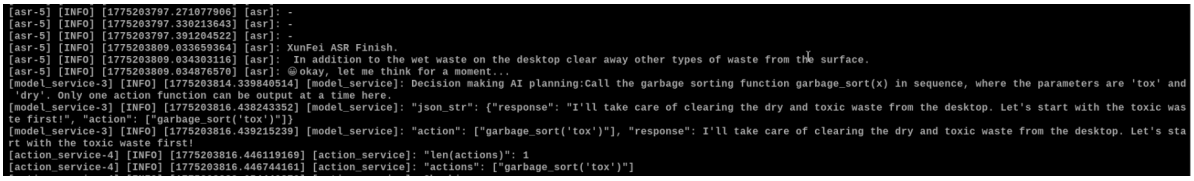
2.2. Leave a certain type of garbage

2.2.1. Startup

Wake up the voice module and voice input, for example:

```
In addition to the wet waste on the desktop, clear away other types of waste from
the surface.
```

Wait for the large model to think and reply, as shown in the figure below:



Except for the wet garbage label code blocks, the robotic arm will grab other types of garbage label code blocks on the desktop.

2.2.2. Task Planning

Sequentially call the garbage sorting function `garbage_sort(x)` with parameters 'tox', 'rec' and 'dry'. Here only one action function can be output each time.

Task steps:

1. Call `garbage_sort('tox')`
2. Call `garbage_sort('rec')`
3. Call `garbage_sort('dry')`

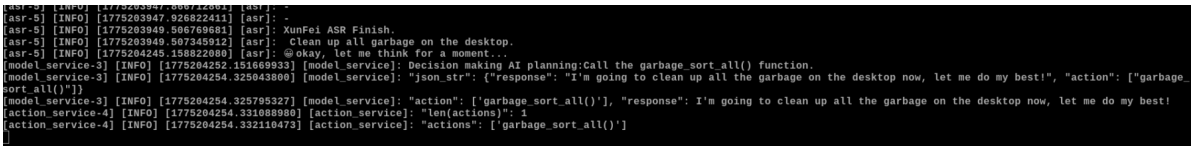
2.3. Sorting all garbage on the desktop

2.3.1. Startup

After waking up the voice module, input the voice command, for example:

```
Clean up all garbage on the desktop
```

Wait for the large model to think and reply, as shown in the figure below:



The robotic arm will sort all the garbage label blocks on the desktop.

2.3.2. Task Planning

Call the `garbage_sort_all()` function.